

Esraaj-Lab15

May 10, 2026

1 Lab 15 - Human Activity Recognition using LSTM

Esraaj Sarkar Gupta, on the 10th of May, 2026

```
[4]: # ---- Imports and Helpers ---- #
import os
import numpy as np
from numpy import mean, std, dstack
from pandas import read_csv
from matplotlib import pyplot as plt

from keras.models import Sequential
from keras.layers import Dense, Flatten, Dropout, LSTM
from keras.utils import to_categorical

print(f"Completed all imports successfully...")

# Is this the last lab already??
# I will miss talking to you every week ike this, Indrani Ma'am.
# I hope you TA another course for me in the future where I can can submit labs
↳to you again!
```

Completed all imports successfully...

```
[ ]: # ---- Dataset Loading Functions ---- #

def load_file(filepath):
    """Load a single file as a numpy array."""
    dataframe = read_csv(filepath, header=None, sep="\s+") # Whitespace arg is
↳deprecated
    return dataframe.values

def load_group(filenamees, prefix=''):
    """Load a list of files into a 3D array of [samples, timesteps, features].
    ↳"""
    loaded : list = list([])

    for name in filenamees:
```

```

        data = load_file(prefix + name)
        loaded.append(data)

        # stack group so that features are in 3D
        loaded = dstack(loaded)
        return loaded

def load_dataset_group(group, prefix=''):
    """Load a dataset group, such as train or test."""

    filepath = prefix + group + '/Inertial Signals/'

    # Load all the files as a single array
    filenames = list([])

    # Total acceleration
    filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt',
        ↪ 'total_acc_z_'+group+'.txt']

    # Body acceleration
    filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt',
        ↪ 'body_acc_z_'+group+'.txt']

    # Body gyroscope
    filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt',
        ↪ 'body_gyro_z_'+group+'.txt']

    # load input data
    X = load_group(filenames, filepath)
    # load class output
    y = load_file(prefix + group + '/y_'+group+'.txt')
    return X, y

def load_dataset(prefix=''):
    """Load the dataset, returns train and test X and y elements."""

    # Training dataset
    trainX, trainy = load_dataset_group('train', prefix + 'UCI HAR Dataset/')

    # Testing dataset
    testX, testy = load_dataset_group('test', prefix + 'UCI HAR Dataset/')

    # Zero-offset class values
    trainy = trainy - 1
    testy = testy - 1

    # One-Hot encoding

```

```

trainy = to_categorical(trainy)
testy = to_categorical(testy)

print(f"Dataset loaded. Train shape: {trainX.shape}, Test shape: {testX.
↪shape}")

return trainX, trainy, testX, testy

```

```

<>:5: SyntaxWarning: invalid escape sequence '\s'
<>:5: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_274395/3928900262.py:5: SyntaxWarning: invalid escape sequence
'\s'
dataframe = read_csv(filepath, header=None, sep="\s+")

```

```

[ ]: # ---- Model Evaluation & 10-Iteration Run ---- #
def evaluate_model(trainX, trainy, testX, testy):
    """
    Define, compile, fit, and evaluate the LSTM model.
    """

    # -- Run Settings -- #
    verbose = 1      # I need to know what's going on for 30 minutes in this ↪
↪machine
    epochs = 15      # I only have 30 minutes
    batch_size = 64

    # Setting dimensions
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], ↪
↪trainy.shape[1]

    # Define Model
    model = Sequential()
    model.add(LSTM(100, input_shape=(n_timesteps, n_features)))
    model.add(Dropout(0.5))
    model.add(Dense(100, activation='relu'))
    model.add(Dense(n_outputs, activation='softmax'))

    # Compile Model
    model.compile(loss='categorical_crossentropy', optimizer='adam', ↪
↪metrics=['accuracy'])

    # Fit Model (as the function name would suggest)
    model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, ↪
↪verbose=verbose)

    # Evaluate model
    _, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)

```

```

    return accuracy

def run_experiment(repeats=10):
    """
    Run the model evaluation multiple times and report results.
    """
    trainX, trainy, testX, testy = load_dataset()

    scores = list()
    for r in range(repeats):
        print(f'\n--- Starting Run {r+1} ---')

        score = evaluate_model(trainX, trainy, testX, testy)
        score = score * 100.0

        print(f'> Run {r+1} Complete: Accuracy = {score:.3f}%')

        scores.append(score)

    print(f'\nFinal Results over {repeats} runs:')
    print(f'Mean Accuracy: {mean(scores):.3f}%')
    print(f'Standard Deviation: {std(scores):.3f}%')

# Execute the 10 runs
run_experiment()

# The output will be a little long, please bear with me.

```

Dataset loaded. Train shape: (7352, 128, 9), Test shape: (2947, 128, 9)

--- Starting Run 1 ---

Epoch 1/15

```

/home/esraaj/jupyterenv/lib/python3.13/site-
packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)

```

115/115 9s 71ms/step -

accuracy: 0.4823 - loss: 1.2136

Epoch 2/15

115/115 8s 69ms/step -

accuracy: 0.6459 - loss: 0.7935

Epoch 3/15

115/115 8s 70ms/step -

accuracy: 0.7021 - loss: 0.7548

Epoch 4/15

```

115/115          8s 69ms/step -
accuracy: 0.6926 - loss: 0.7967
Epoch 5/15
115/115          8s 70ms/step -
accuracy: 0.7530 - loss: 0.6066
Epoch 6/15
115/115          7s 58ms/step -
accuracy: 0.8327 - loss: 0.4316
Epoch 7/15
115/115          7s 59ms/step -
accuracy: 0.8995 - loss: 0.2944
Epoch 8/15
115/115          7s 62ms/step -
accuracy: 0.9236 - loss: 0.2197
Epoch 9/15
115/115          7s 63ms/step -
accuracy: 0.9193 - loss: 0.2229
Epoch 10/15
115/115          7s 63ms/step -
accuracy: 0.9272 - loss: 0.1975
Epoch 11/15
115/115          7s 64ms/step -
accuracy: 0.9392 - loss: 0.1635
Epoch 12/15
115/115          9s 75ms/step -
accuracy: 0.9444 - loss: 0.1546
Epoch 13/15
115/115          9s 76ms/step -
accuracy: 0.9387 - loss: 0.1478
Epoch 14/15
115/115          9s 75ms/step -
accuracy: 0.9404 - loss: 0.1521
Epoch 15/15
115/115          9s 75ms/step -
accuracy: 0.9433 - loss: 0.1419
> Run 1 Complete: Accuracy = 90.736%

```

--- Starting Run 2 ---

```

Epoch 1/15
115/115          10s 75ms/step -
accuracy: 0.5094 - loss: 1.1540
Epoch 2/15
115/115          9s 75ms/step -
accuracy: 0.6692 - loss: 0.7481
Epoch 3/15
115/115          9s 75ms/step -
accuracy: 0.7594 - loss: 0.5762
Epoch 4/15

```

```

115/115          9s 75ms/step -
accuracy: 0.8353 - loss: 0.4319
Epoch 5/15
115/115          9s 75ms/step -
accuracy: 0.8341 - loss: 0.4748
Epoch 6/15
115/115          9s 76ms/step -
accuracy: 0.8203 - loss: 0.5105
Epoch 7/15
115/115          9s 76ms/step -
accuracy: 0.9095 - loss: 0.2589
Epoch 8/15
115/115          9s 76ms/step -
accuracy: 0.9334 - loss: 0.1856
Epoch 9/15
115/115          9s 75ms/step -
accuracy: 0.9373 - loss: 0.1592
Epoch 10/15
115/115          9s 76ms/step -
accuracy: 0.9129 - loss: 0.2446
Epoch 11/15
115/115          9s 75ms/step -
accuracy: 0.8698 - loss: 0.3411
Epoch 12/15
115/115          9s 77ms/step -
accuracy: 0.9276 - loss: 0.1958
Epoch 13/15
115/115          9s 76ms/step -
accuracy: 0.9397 - loss: 0.1567
Epoch 14/15
115/115          9s 76ms/step -
accuracy: 0.9359 - loss: 0.1641
Epoch 15/15
115/115          9s 76ms/step -
accuracy: 0.9426 - loss: 0.1456
> Run 2 Complete: Accuracy = 90.159%

```

--- Starting Run 3 ---

```

Epoch 1/15
115/115          10s 75ms/step -
accuracy: 0.4805 - loss: 1.2537
Epoch 2/15
115/115          9s 76ms/step -
accuracy: 0.6133 - loss: 0.9079
Epoch 3/15
115/115          9s 77ms/step -
accuracy: 0.6989 - loss: 0.6779
Epoch 4/15

```

```

115/115          9s 78ms/step -
accuracy: 0.7695 - loss: 0.5685
Epoch 5/15
115/115          9s 78ms/step -
accuracy: 0.8730 - loss: 0.3477
Epoch 6/15
115/115          9s 78ms/step -
accuracy: 0.9181 - loss: 0.2431
Epoch 7/15
115/115          9s 77ms/step -
accuracy: 0.9306 - loss: 0.1824
Epoch 8/15
115/115          9s 77ms/step -
accuracy: 0.9411 - loss: 0.1525
Epoch 9/15
115/115          9s 78ms/step -
accuracy: 0.9244 - loss: 0.2047
Epoch 10/15
115/115          9s 78ms/step -
accuracy: 0.9459 - loss: 0.1415
Epoch 11/15
115/115          9s 79ms/step -
accuracy: 0.9463 - loss: 0.1309
Epoch 12/15
115/115          9s 74ms/step -
accuracy: 0.9494 - loss: 0.1271
Epoch 13/15
115/115          8s 73ms/step -
accuracy: 0.9264 - loss: 0.1941
Epoch 14/15
115/115          9s 74ms/step -
accuracy: 0.9506 - loss: 0.1261
Epoch 15/15
115/115          9s 76ms/step -
accuracy: 0.9476 - loss: 0.1286
> Run 3 Complete: Accuracy = 90.159%

```

--- Starting Run 4 ---

```

Epoch 1/15
115/115          9s 68ms/step -
accuracy: 0.5158 - loss: 1.1295
Epoch 2/15
115/115          8s 68ms/step -
accuracy: 0.6670 - loss: 0.7694
Epoch 3/15
115/115          8s 70ms/step -
accuracy: 0.7533 - loss: 0.5869
Epoch 4/15

```

```

115/115          8s 69ms/step -
accuracy: 0.8637 - loss: 0.3777
Epoch 5/15
115/115          8s 71ms/step -
accuracy: 0.9246 - loss: 0.2091
Epoch 6/15
115/115          8s 70ms/step -
accuracy: 0.9300 - loss: 0.1871
Epoch 7/15
115/115          8s 72ms/step -
accuracy: 0.9385 - loss: 0.1581
Epoch 8/15
115/115          8s 72ms/step -
accuracy: 0.9238 - loss: 0.2203
Epoch 9/15
115/115          9s 75ms/step -
accuracy: 0.9274 - loss: 0.2118
Epoch 10/15
115/115          9s 79ms/step -
accuracy: 0.9446 - loss: 0.1423
Epoch 11/15
115/115          9s 78ms/step -
accuracy: 0.9475 - loss: 0.1322
Epoch 12/15
115/115          9s 78ms/step -
accuracy: 0.9460 - loss: 0.1439
Epoch 13/15
115/115          9s 78ms/step -
accuracy: 0.9506 - loss: 0.1239
Epoch 14/15
115/115          9s 78ms/step -
accuracy: 0.9463 - loss: 0.1319
Epoch 15/15
115/115          9s 78ms/step -
accuracy: 0.9382 - loss: 0.1554
> Run 4 Complete: Accuracy = 86.664%

```

--- Starting Run 5 ---

```

Epoch 1/15
115/115          10s 77ms/step -
accuracy: 0.5170 - loss: 1.1524
Epoch 2/15
115/115          9s 77ms/step -
accuracy: 0.6893 - loss: 0.7664
Epoch 3/15
115/115          9s 79ms/step -
accuracy: 0.7658 - loss: 0.5846
Epoch 4/15

```



```

115/115          9s 80ms/step -
accuracy: 0.8141 - loss: 0.4758
Epoch 5/15
115/115          9s 80ms/step -
accuracy: 0.8492 - loss: 0.3832
Epoch 6/15
115/115          9s 80ms/step -
accuracy: 0.9093 - loss: 0.2782
Epoch 7/15
115/115          9s 80ms/step -
accuracy: 0.9300 - loss: 0.2057
Epoch 8/15
115/115          9s 81ms/step -
accuracy: 0.9286 - loss: 0.1931
Epoch 9/15
115/115          10s 83ms/step -
accuracy: 0.9414 - loss: 0.1541
Epoch 10/15
115/115          9s 82ms/step -
accuracy: 0.9388 - loss: 0.1516
Epoch 11/15
115/115          9s 82ms/step -
accuracy: 0.9448 - loss: 0.1405
Epoch 12/15
115/115          9s 81ms/step -
accuracy: 0.9468 - loss: 0.1328
Epoch 13/15
115/115          10s 83ms/step -
accuracy: 0.9470 - loss: 0.1230
Epoch 14/15
115/115          9s 82ms/step -
accuracy: 0.9531 - loss: 0.1199
Epoch 15/15
115/115          9s 81ms/step -
accuracy: 0.9501 - loss: 0.1254
> Run 5 Complete: Accuracy = 89.684%

```

--- Starting Run 6 ---

```

Epoch 1/15
115/115          12s 88ms/step -
accuracy: 0.5182 - loss: 1.1625
Epoch 2/15
115/115          10s 89ms/step -
accuracy: 0.6857 - loss: 0.7608
Epoch 3/15
115/115          10s 90ms/step -
accuracy: 0.7727 - loss: 0.5871
Epoch 4/15

```

```

115/115          10s 90ms/step -
accuracy: 0.8550 - loss: 0.4117
Epoch 5/15
115/115          11s 91ms/step -
accuracy: 0.8841 - loss: 0.3271
Epoch 6/15
115/115          11s 91ms/step -
accuracy: 0.9159 - loss: 0.2399
Epoch 7/15
115/115          11s 93ms/step -
accuracy: 0.9252 - loss: 0.2108
Epoch 8/15
115/115          11s 92ms/step -
accuracy: 0.9353 - loss: 0.1792
Epoch 9/15
115/115          11s 92ms/step -
accuracy: 0.9433 - loss: 0.1485
Epoch 10/15
115/115          11s 91ms/step -
accuracy: 0.9450 - loss: 0.1525
Epoch 11/15
115/115          11s 93ms/step -
accuracy: 0.9486 - loss: 0.1272
Epoch 12/15
115/115          11s 93ms/step -
accuracy: 0.9490 - loss: 0.1250
Epoch 13/15
115/115          11s 93ms/step -
accuracy: 0.9257 - loss: 0.1995
Epoch 14/15
115/115          11s 94ms/step -
accuracy: 0.9410 - loss: 0.1613
Epoch 15/15
115/115          11s 95ms/step -
accuracy: 0.9489 - loss: 0.1288
> Run 6 Complete: Accuracy = 91.415%

```

--- Starting Run 7 ---

```

Epoch 1/15
115/115          13s 96ms/step -
accuracy: 0.5065 - loss: 1.1576
Epoch 2/15
115/115          11s 95ms/step -
accuracy: 0.6655 - loss: 0.8106
Epoch 3/15
115/115          11s 94ms/step -
accuracy: 0.7878 - loss: 0.5360
Epoch 4/15

```

```

115/115          11s 98ms/step -
accuracy: 0.8285 - loss: 0.4707
Epoch 5/15
115/115          12s 102ms/step -
accuracy: 0.8818 - loss: 0.3455
Epoch 6/15
115/115          12s 102ms/step -
accuracy: 0.9072 - loss: 0.2808
Epoch 7/15
115/115          13s 110ms/step -
accuracy: 0.9351 - loss: 0.1891
Epoch 8/15
115/115          13s 109ms/step -
accuracy: 0.9388 - loss: 0.1696
Epoch 9/15
115/115          12s 105ms/step -
accuracy: 0.9464 - loss: 0.1458
Epoch 10/15
115/115          12s 108ms/step -
accuracy: 0.9353 - loss: 0.1653
Epoch 11/15
115/115          14s 123ms/step -
accuracy: 0.9293 - loss: 0.1888
Epoch 12/15
115/115          13s 113ms/step -
accuracy: 0.8789 - loss: 0.3616
Epoch 13/15
115/115          12s 105ms/step -
accuracy: 0.9368 - loss: 0.1753
Epoch 14/15
115/115          12s 105ms/step -
accuracy: 0.9404 - loss: 0.1550
Epoch 15/15
115/115          12s 101ms/step -
accuracy: 0.9472 - loss: 0.1367
> Run 7 Complete: Accuracy = 89.141%

```

--- Starting Run 8 ---

```

Epoch 1/15
115/115          13s 101ms/step -
accuracy: 0.4737 - loss: 1.2393
Epoch 2/15
115/115          12s 103ms/step -
accuracy: 0.6047 - loss: 0.9221
Epoch 3/15
115/115          12s 103ms/step -
accuracy: 0.7263 - loss: 0.6618
Epoch 4/15

```

```

115/115          12s 104ms/step -
accuracy: 0.7667 - loss: 0.5644
Epoch 5/15
115/115          12s 105ms/step -
accuracy: 0.8192 - loss: 0.5203
Epoch 6/15
115/115          12s 108ms/step -
accuracy: 0.8664 - loss: 0.3809
Epoch 7/15
115/115          13s 109ms/step -
accuracy: 0.9177 - loss: 0.2410
Epoch 8/15
115/115          12s 104ms/step -
accuracy: 0.9253 - loss: 0.2066
Epoch 9/15
115/115          12s 106ms/step -
accuracy: 0.9366 - loss: 0.1746
Epoch 10/15
115/115          12s 103ms/step -
accuracy: 0.9214 - loss: 0.2130
Epoch 11/15
115/115          14s 120ms/step -
accuracy: 0.9382 - loss: 0.1723
Epoch 12/15
115/115          13s 114ms/step -
accuracy: 0.9460 - loss: 0.1448
Epoch 13/15
115/115          13s 111ms/step -
accuracy: 0.9436 - loss: 0.1483
Epoch 14/15
115/115          12s 107ms/step -
accuracy: 0.9275 - loss: 0.2179
Epoch 15/15
115/115          10s 91ms/step -
accuracy: 0.9339 - loss: 0.1794
> Run 8 Complete: Accuracy = 87.750%

```

--- Starting Run 9 ---

```

Epoch 1/15
115/115          9s 69ms/step -
accuracy: 0.4997 - loss: 1.1675
Epoch 2/15
115/115          8s 72ms/step -
accuracy: 0.6171 - loss: 0.9260
Epoch 3/15
115/115          9s 79ms/step -
accuracy: 0.7318 - loss: 0.6455
Epoch 4/15

```

```

115/115          9s 79ms/step -
accuracy: 0.7248 - loss: 0.7025
Epoch 5/15
115/115          9s 79ms/step -
accuracy: 0.7922 - loss: 0.5572
Epoch 6/15
115/115          9s 79ms/step -
accuracy: 0.8441 - loss: 0.4320
Epoch 7/15
115/115          9s 81ms/step -
accuracy: 0.9074 - loss: 0.2743
Epoch 8/15
115/115          10s 87ms/step -
accuracy: 0.9264 - loss: 0.2101
Epoch 9/15
115/115          10s 86ms/step -
accuracy: 0.9300 - loss: 0.1839
Epoch 10/15
115/115          10s 87ms/step -
accuracy: 0.9372 - loss: 0.1619
Epoch 11/15
115/115          10s 87ms/step -
accuracy: 0.9338 - loss: 0.1766
Epoch 12/15
115/115          10s 89ms/step -
accuracy: 0.9376 - loss: 0.1592
Epoch 13/15
115/115          10s 84ms/step -
accuracy: 0.9090 - loss: 0.2574
Epoch 14/15
115/115          9s 80ms/step -
accuracy: 0.9373 - loss: 0.1654
Epoch 15/15
115/115          9s 81ms/step -
accuracy: 0.9399 - loss: 0.1568
> Run 9 Complete: Accuracy = 89.379%

```

--- Starting Run 10 ---

```

Epoch 1/15
115/115          10s 80ms/step -
accuracy: 0.4815 - loss: 1.2473
Epoch 2/15
115/115          9s 79ms/step -
accuracy: 0.6513 - loss: 0.8214
Epoch 3/15
115/115          9s 79ms/step -
accuracy: 0.7319 - loss: 0.6156
Epoch 4/15

```

```

115/115          9s 76ms/step -
accuracy: 0.8251 - loss: 0.4736
Epoch 5/15
115/115          9s 76ms/step -
accuracy: 0.8811 - loss: 0.3478
Epoch 6/15
115/115          9s 75ms/step -
accuracy: 0.9174 - loss: 0.2487
Epoch 7/15
115/115          9s 74ms/step -
accuracy: 0.9067 - loss: 0.2652
Epoch 8/15
115/115          9s 75ms/step -
accuracy: 0.9203 - loss: 0.2127
Epoch 9/15
115/115          9s 75ms/step -
accuracy: 0.9040 - loss: 0.2705
Epoch 10/15
115/115          9s 78ms/step -
accuracy: 0.9402 - loss: 0.1567
Epoch 11/15
115/115          9s 79ms/step -
accuracy: 0.9472 - loss: 0.1461
Epoch 12/15
115/115          9s 79ms/step -
accuracy: 0.9444 - loss: 0.1478
Epoch 13/15
115/115          9s 76ms/step -
accuracy: 0.9407 - loss: 0.1658
Epoch 14/15
115/115          9s 78ms/step -
accuracy: 0.9464 - loss: 0.1446
Epoch 15/15
115/115          9s 79ms/step -
accuracy: 0.9476 - loss: 0.1293
> Run 10 Complete: Accuracy = 90.465%

```

Final Results over 10 runs:
 Mean Accuracy: 89.555%
 Standard Deviation: 1.351

2 Report

Max CPU Temp 96C Runtime 24m 36s

Question 1 Human Activity Recognition uses continuous sensor data collected over time. Sequential time series data such as this show strong temporal dependencies between measurements.

For this reason, we use LSTMs that, unlike SVMs or Random Forest Classifiers, do not treat each measurement as an independent data point. LSTMs are capable of learning from sequential, temporal dynamics.

Question 2 This is a many-to-one scenario, since many time steps (128 of them from 3D sensor data) are used to output a single classification to represent the state of the subject during the period of data collection.

Question 3 In the context of LSTMs, the Long-Term Memory is represented by the cell state C_t . The cell state is a vector that represents the internal memory of the unit. It is modified only by linear interactions (such as multiplication and addition) through the forget and input gates. Since the gradient may flow through this state with very little modification, information from many time steps is preserved. This allows the preservation of a stable representation of the global context; hence “long-term” memory. On the other hand, Short-Term Memory is given by the hidden state h_t . It is calculated by passing the current cell state through a non-linear activation function (such as tanh) and filtering it with the output gate. This contains information relevant to the local context and used to compute the prediction. Unlike the cell state, the hidden state is volatile and changes significantly at every step of the sequence.

Question 4 An LSTM node has three primary parts or “gates”.

- The Forget Gate: This gate decides which information from the previous long-term memory (cell state) should be discarded or kept. It takes the current input and the previous short-term memory (hidden state) and outputs a value between 0 and 1 for each number in the cell state as a binary decision for retention or discarding.
- Input Gate: This gate determines what new information from the current time step is relevant and should be added onto the long-term memory. It involves a two-step process where a sigmoid layer decides which values to update and a non-linear tanh layer creates a vector of new candidate values that could be added to the cell state.
- Output Gate: This gate determines the next short-term memory (hidden state). It filters the newly updated cell state based on the current input and the previous hidden state to ensure only the relevant information is passed on to the next node and used for the final prediction.

Question 5 Traditional RNNs suffer heavily from the vanishing gradient problem. When dealing with long data sequences, the error gradients used to update the weights shrink exponentially as they propagate backward through time. This makes it almost impossible for standard RNNs to learn long-term dependencies. LSTMs fix this via the cell state and the gating mechanisms. This allows error signals to flow relatively unchanged over multiple time steps, effectively bridging long temporal gaps.